Chapter 2 – section 4

Java Arrays

Java Arrays

- Array is an object / data structure that stores multiple variables of the same type.
- Array is an ordered collection of values with two distinguishing characters:
  - ✓Ordered and fixed length.
  - ✓Homogeneous: Every value in the array must be of the same type.
- The individual values in an array are called **elements**.
- The number of elements is called the **length** of the array.
- Each element is identified by its position number in the array, which is called **index**. In Java, the index numbers begin with 0 and end with **arrayLength-1**.

Declaring Arrays

- Syntax:

  datatype[] arrayRefVar;

  Example:

  double[] myList;

  // The following style is also allowed, but not preferred

  datatype arrayRefVar[];

  Example:

  double myList[];

Creating Arrays

- Arrays are created using the **new** keyword.

  arrayRefVar = new datatype[arraySize];

  Example:

  myList = new double[10];

- We may also declare more than one array in a single statement, as shown below:

  int[] a, b, c;

- While the array is a reference type, the elements of the array are not of reference type.
  - ✓ **a[]** is a reference type while **a[0]** is a primitive type.

Declaring and Creating in One Step

☞ **datatype[] arrayRefVar = new datatype[arraySize];**

example: double[] myList = new double[10];

☞ **datatype arrayRefVar[] = new datatype[arraySize];**

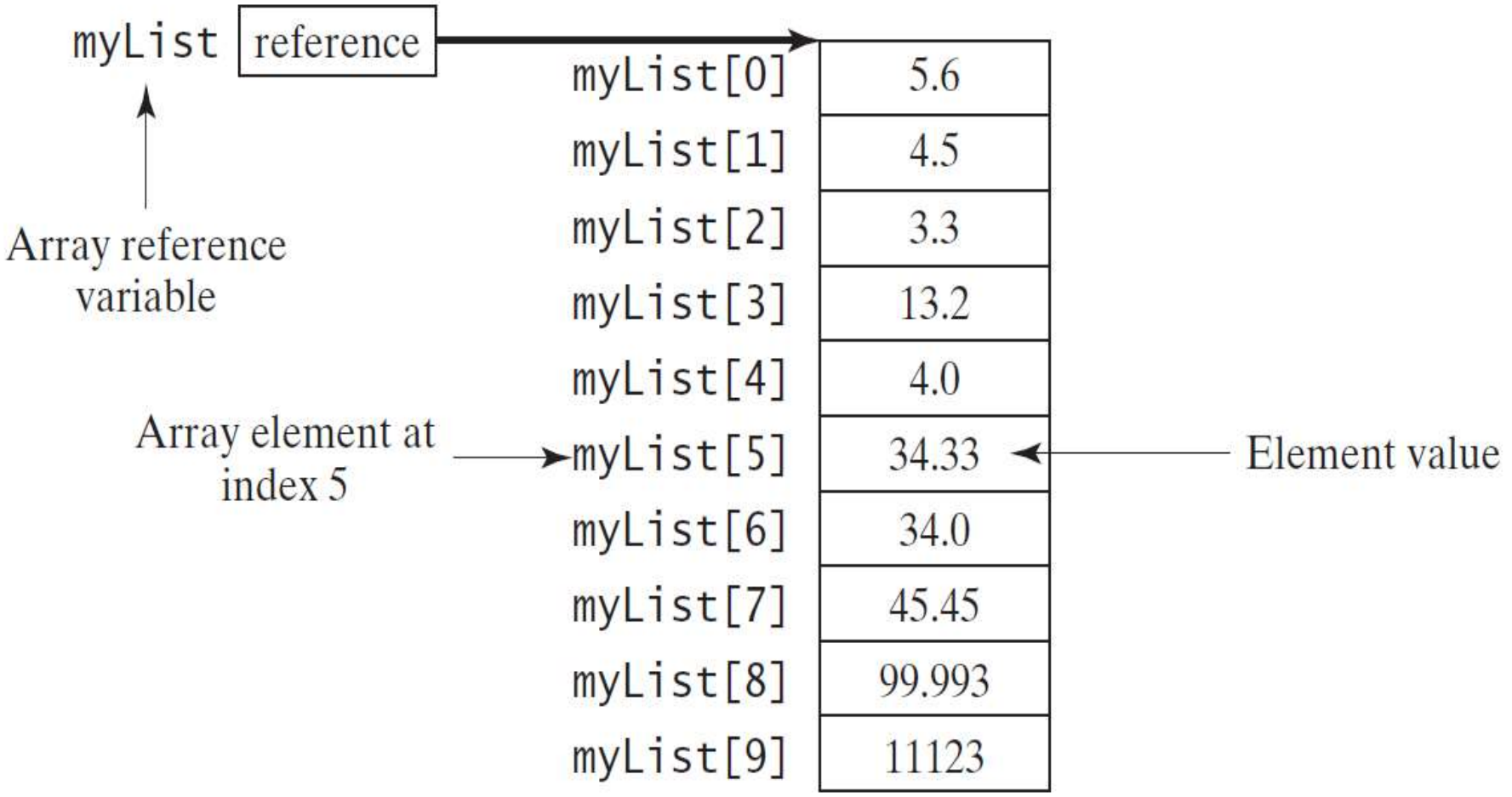
example: double myList[] = new double[10];

➢ To initialize an array, we use the syntax:

arrayRefVar[index] = value;

Cont'd...

```
double[] myList = new double[10];
```



Array Size and Default Values

- Once an array is created, its size is fixed.
- You can find its size using:

  arrayRefVar.length

  For example,

  myList.length returns 10

  - When an array is created, its elements are assigned the default value.
  - The array elements are accessed through the index.

arrayRefVar[index];

Array Initializer

- Declaring, creating and initializing in one step:

  double[] myList = {1.9, 2.9, 3.4, 3.5};

- This shorthand notation is equivalent to the following statements:

```
double[] myList = new double[4];
myList[0] = 1.9;
myList[1] = 2.9;
myList[2] = 3.4;
myList[3] = 3.5;
```

Array Initializer…

CAUTION!

- Using the shorthand notation, you have to declare, create, and initialize the array all in one statement.
- Splitting it would cause a syntax error.

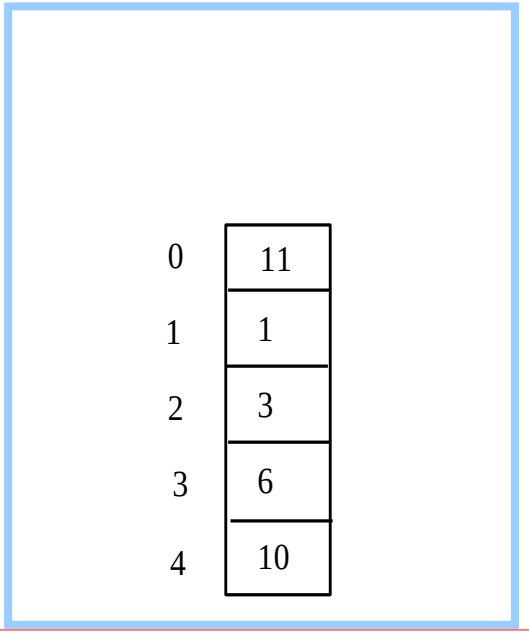
For example, the following is wrong:

```
double[] myList;

myList = {1.9, 2.9, 3.4, 3.5};
```

Array Example

```
public class Test {
  public static void main(String[] args) {
    int[] values = new int[5];
    for (int i = 1; i < 5; i++) {
      values[i] = i + values[i-1];
    }
    values[0] = values[1] + values[4];
  }
}
```



- Note: Accessing an array out of bounds is a common programming error that throws a runtime **ArrayIndexOutOfBoundsException**.
- To avoid it, make sure that you do not use an index beyond arrayRefVar.length – 1.

Processing Arrays

- When processing array elements, you will often use a for loop
- The following are some examples of processing arrays.

1. Initializing arrays with input values
2. Initializing arrays with random values
3. Printing arrays
4. Summing all elements
5. Finding the largest element
6. Finding the smallest index of the largest element

Initializing arrays with input and random values

Assume the array is created as follows:

```
double[] myList =new double[10];
```

```
//initialize with input values
java.util.Scanner input = new java.util.Scanner(System.in);
System.out.print("Enter " + myList.length + " values: ");
for (int i = 0; i < myList.length; i++)
  myList[i] = input.nextDouble();

//initialize with random values
for (int i = 0; i < myList.length; i++) {
  myList[i] = Math.random() * 100;
}
```

Printing arrays and summing all elements

```
// Printing arrays
for (int i = 0; i < myList.length; i++) {
  System.out.print(myList[i] + " ");
}
```

- For an array of the char[] type, it can be printed using one print statement. For example, the following code displays Ethiopia:

```
char[] country = {‘E‘, ‘t‘, ‘h‘, ‘i‘, ‘o‘, ’p‘, ’i’, ’a’};
System.out.println(country);
```

Finding the largest element

```
double max = myList[0];
for (int i = 1; i < myList.length; i++) {
  if (myList[i] > max)
  max = myList[i];
}
```

Enhanced for Loop

- Used to traverse the complete array sequentially without using an index variable.
- The syntax is:

```
for(elementType value: arrayRefVar)
```

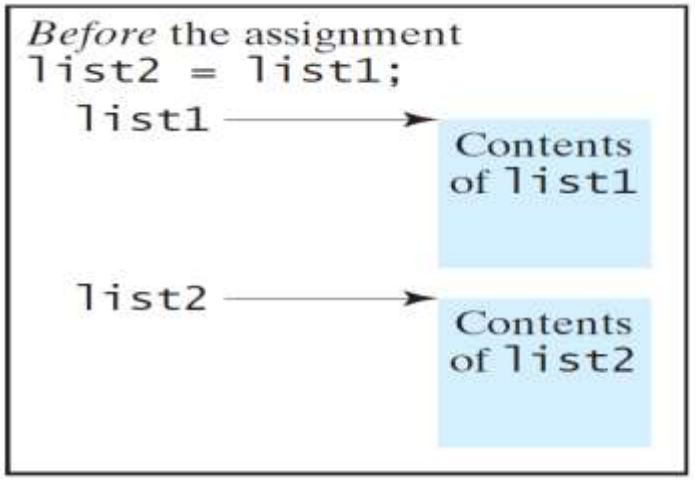
For example, the following code displays all elements in the array myList:

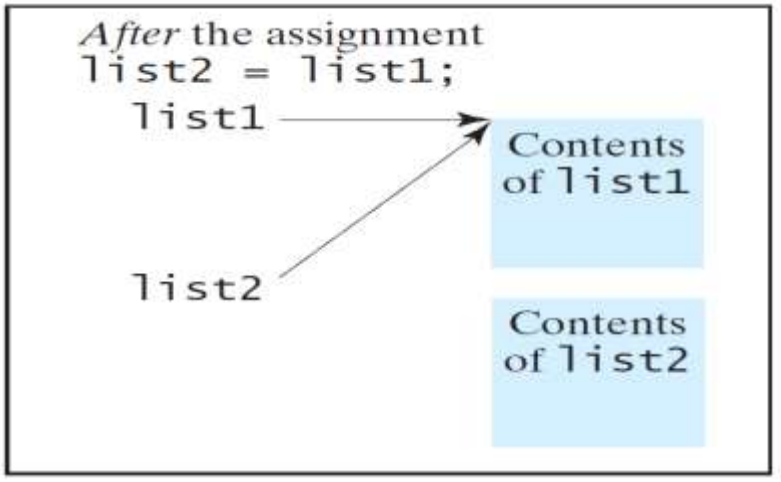
```
for(double value: myList)
   System.out.println(value);
```

Copying Arrays

// Using the assignment statement (=):

list2 = list1;





//Using a loop:

```
int[] sourceArray = {2, 3, 1, 5, 10};
int[] targetArray = new int[sourceArray.length};
for (int i = 0; i < sourceArray.length; i++)
  targetArray[i] = sourceArray[i];
```

The `arraycopy` method

- Another approach is to use the **arraycopy** method in the **java.lang.System** class to copy arrays.

  arraycopy(sourceArray, src_pos, targetArray, tar_pos, length);

Example:

```
System.arraycopy(sourceArray, 0, targetArray, 0,
sourceArray.length);
```

Array of Objects

- Arrays are also capable of storing objects.
- To create an array of objects the syntax is:

  classType[] refVariable = new classType[size];

  Example: Student[] studentArray = new Student[7];
- The above statement creates the array which can hold references to seven Student objects in seven memory spaces.
- It doesn't create the Student objects themselves.
- They have to be created separately using the constructor of the Student class.

Array of Objects…

- If we try to access the Student objects even before creating them, run time errors would occur.
- For instance, the following statement:
  studentArray[0].marks = 100;
  - ☞ throws a **NullPointerException** during runtime which indicates that studentArray[0] isn't yet pointing to a Student object.
- The Student objects have to be instantiated using the constructor of the Student class and their references should be assigned to the array elements:

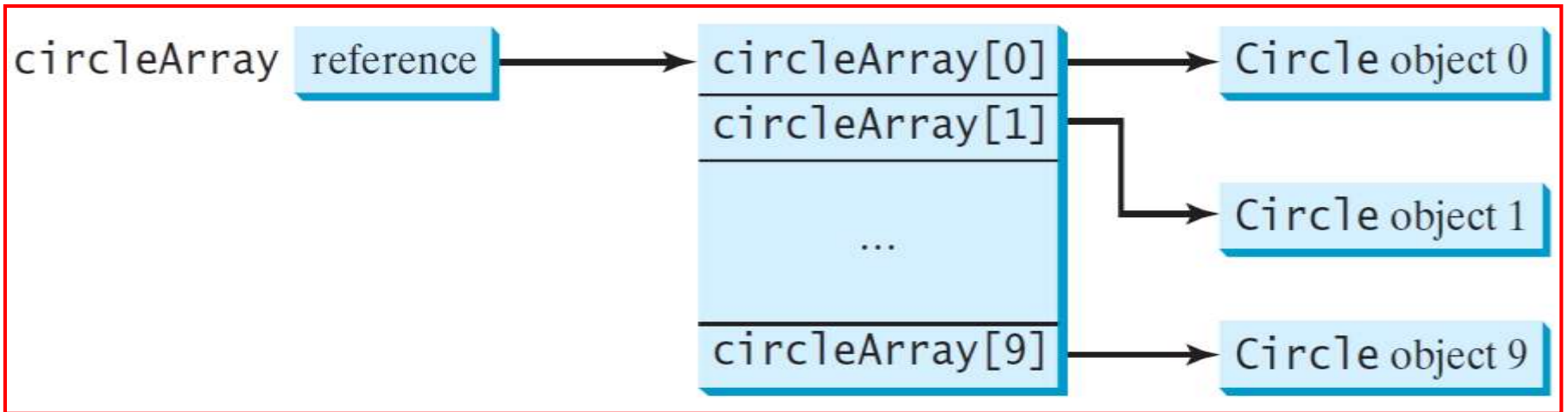
  studentArray[0] = new Student();
- In this way, we can also create the other Student objects.

Array of Objects...

- An array of objects is actually an array of reference variables.

```
Circle[] circleArray = new Circle[10];
```

- So invoking **circleArray[1].getArea()** involves two levels of referencing as shown in the next figure:
  - **circleArray** references to the entire array.
  - **circleArray[1]** references to a Circle object.



Passing Arrays to Methods

- When passing an array to a method, the copy of reference of the array is passed to the method.

```
public static void printArray(int[] array) {
        for (int i = 0; i < array.length; i++) {
        System.out.print(array[i] + " ");
        }
}
```

```
//Invoke the method
int[] list = {3, 1, 2, 6, 4, 2};
printArray(list);
```

```
//Invoke the method
printArray(new int[]{3, 1, 2, 6, 4, 2});
```

Anonymous Array

- The statement

```
printArray(new int[]{3, 1, 2, 6, 4, 2});
```

creates an array using the following syntax:

```
new dataType[]{value0, value1, ..., valuek};
```

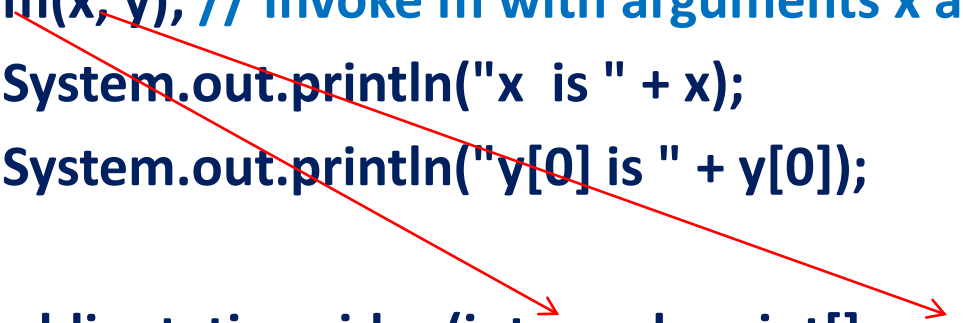
- There is no explicit reference variable for the array. Such array is called an *anonymous array*.

Pass By Value

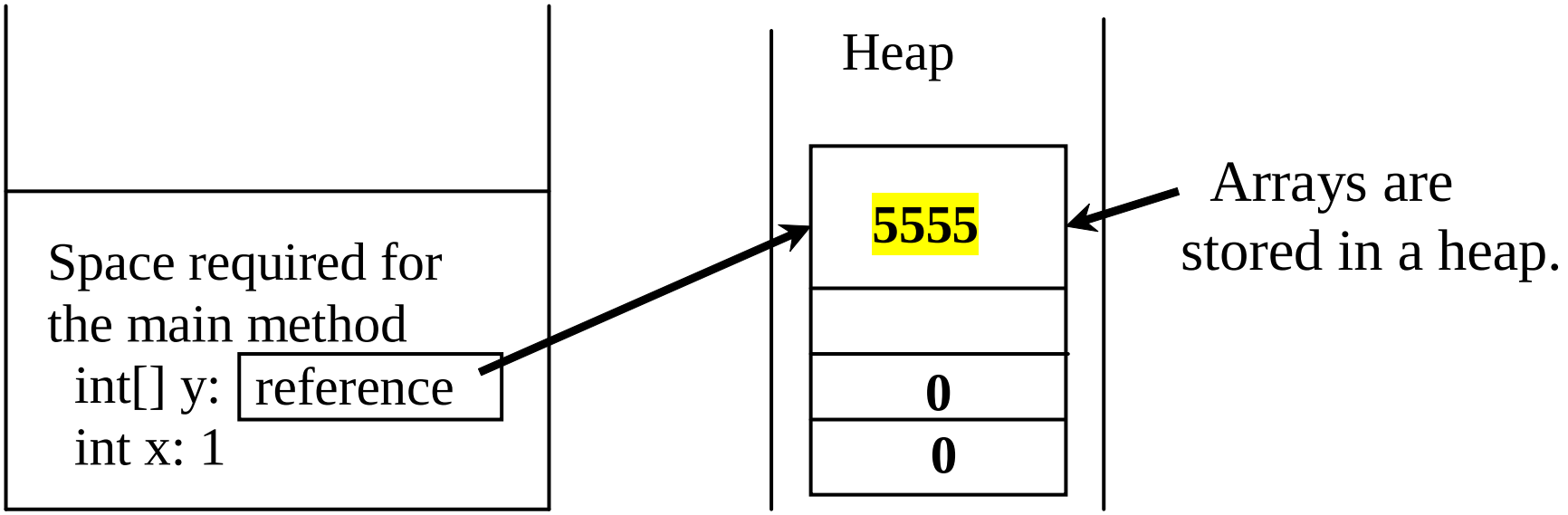
- For a parameter of a primitive type value, the actual value is passed.
  - ✓Changing the value of the local parameter inside the method does not affect the value of the variable outside the method.
- For a parameter of an array type, the value of the parameter contains a reference to an array; this reference is passed to the method.
  - ✓Any changes to the array that occur inside the method body will affect the original array that was passed as the argument.

Simple Example

```
public class Test {
  public static void main(String[] args) {
    int x = 1;
    int[] y = new int[10];
    m(x, y); // Invoke m with arguments x and y
    System.out.println("x  is " + x);
    System.out.println("y[0] is " + y[0]);
  }
  public static void m(int number, int[] numbers) {
    number = 1001; // Assign a new value to number
    numbers[0] = 5555; // Assign a new value to numbers[0]
  }
}
```



Heap



- Arrays are objects in java, so the JVM stores the array in an area of memory, called *heap*, which is used for dynamic memory allocation where blocks of memory are allocated and freed in an arbitrary order.

Returning an Array from a Method

- When a method returns an array, the reference of the array is returned.
- For example, a method which returns a double type array will have the following header:

public double[] methodName ()

Multidimensional Arrays

- Java does not support multidimensional arrays.
- In Java, you can create arrays that contain other arrays.
- An array can hold references to objects and an array itself is a reference type so an array of arrays seems quite possible.
- These are referred to as multidimensional arrays. The syntax for declaring a two-dimensional array is:

  elementType[][] arrayRefVar;

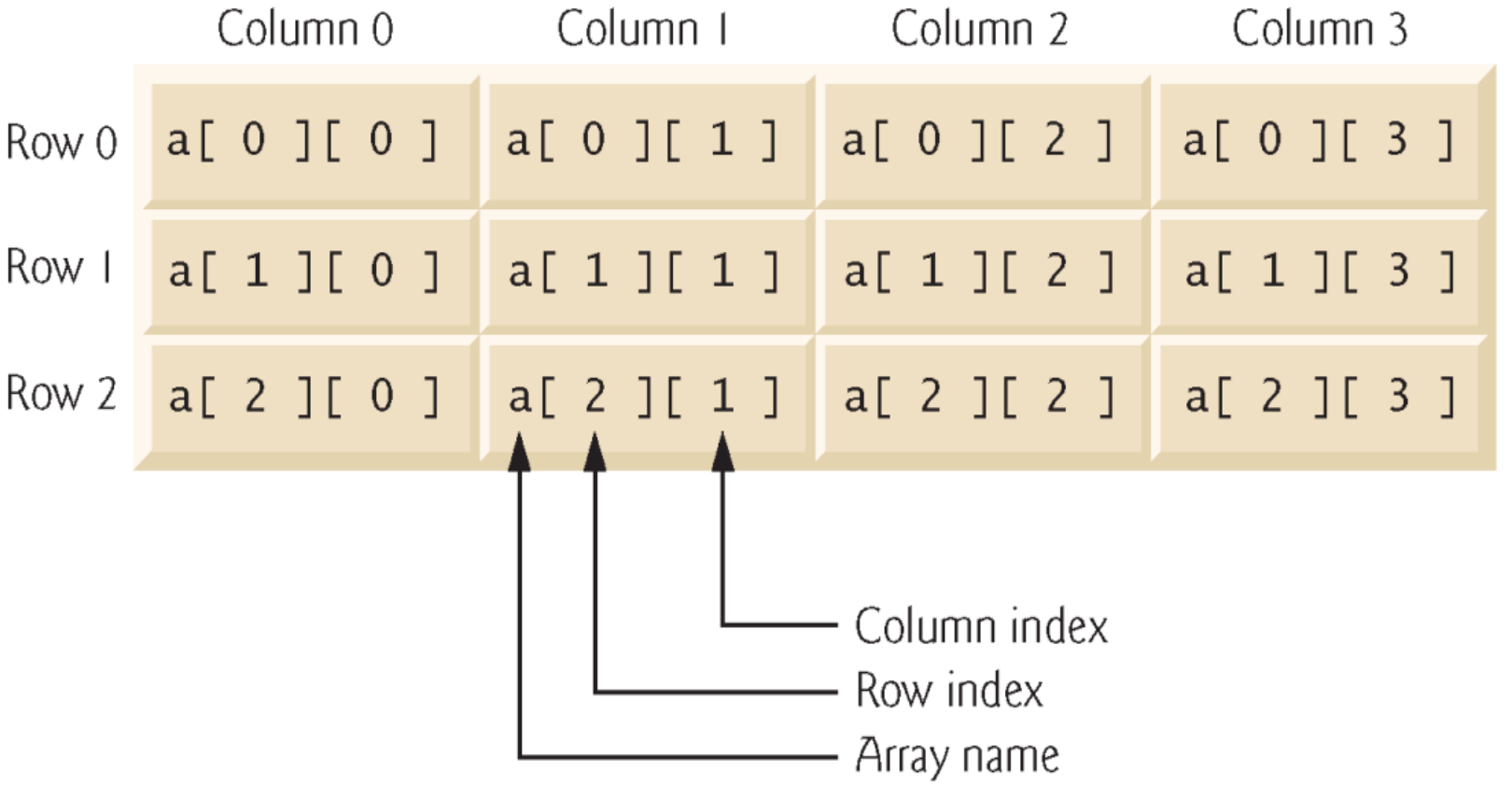
  Or

  elementType arrayRefVar[][]; // Allowed, but not preferred

- The following statement creates a two-dimensional array of integers, which contains 3 arrays containing 4 integers each.

  int[][] a=new int[3][4];

Multidimensional Arrays…



Two-dimensional array with three rows and four columns.

Multidimensional Arrays…

- A two dimensional array can be initialized using an array initializer in the following way.

int[][] a = { { 1,5,74,2}, {4,68,45,65},{5,0,34,54}};

- The above array **a[][]** contains three arrays

{ 1,5,74,2}, {4,68,45,65} and {5,0,34,54}.

- We can create 2D arrays which have different number of elements in its sub arrays(**ragged array**).

int[][] c = { { 4,56,7}, {1}, {45,78} };

Multidimensional Arrays…

- We can also create such non symmetric arrays using the new keyword in the following way:

```
int[][] b = new int[3][];
b[0]=new int[3];
b[1]=new int[1];
b[2]=new int[2];
```

- **b[][]** is reference type and so are b[0], b[1] and b[2].
- However, b[0][0], b[0][1] .... are of primitive type int.
- The length of a two dimensional or a multi-dimensional array gives the number of arrays it contains.
- ✓ For example: b.length gives 3 while b[0].length gives 3, b[1].length gives 1 and so on.

Multidimensional Arrays…

- We manipulate multi-dimensional arrays using nested loops.

  For example:

```
        for ( int i=0; i < a.length;  i++) {
    for(int j=0; j< a[i].length; j++)
       System.out.print ( a[i][j] );
       System.out.println();
}
```

Exercise:
Practice all array processing on Two-Dimensional Arrays which we did on one dimensional arrays

Variable Length Argument Lists

- Variable length argument lists:
  - Can be used to create methods that receive an unspecified number of arguments.
  - Parameter type followed by an ellipsis (…) indicates that the method receives a variable number of arguments of that particular type.
  - The ellipsis can occur only once at the end of a parameter list.

✓ Interpretation: an array whose elements are all of the same type declared before the ellipsis.

```
1  // Fig. 7.20: VarargsTest.java
2  // Using variable-length argument lists.
3
4  public class VarargsTest
5  {
6     // calculate average
7     public static double average(double... numbers)
8     {
9        double total = 0.0;
10
11       // calculate total using the enhanced for statement
12       for (double d : numbers)
13          total += d;
14
15       return total / numbers.length;
16    }
17
18    public static void main(String[] args)
19    {
20       double d1 = 10.0;
21       double d2 = 20.0;
22       double d3 = 30.0;
23       double d4 = 40.0;
24
```

Fig. 7.20 | Using variable-length argument lists. (Part 1 of 2.)

```
25          System.out.printf("d1 = %.1f%nd2 = %.1f%nd3 = %.1f%nd4 = %.1f%n%n",
26             d1, d2, d3, d4);
27
28          System.out.printf("Average of d1 and d2 is %.1f%n",
29             average(d1, d2) );
30          System.out.printf("Average of d1, d2 and d3 is %.1f%n",
31             average(d1, d2, d3) );
32          System.out.printf("Average of d1, d2, d3 and d4 is %.1f%n",
33             average(d1, d2, d3, d4) );
34       }
35    } // end class VarargsTest
```

```
d1 = 10.0
d2 = 20.0
d3 = 30.0
d4 = 40.0

Average of d1 and d2 is 15.0
Average of d1, d2 and d3 is 20.0
Average of d1, d2, d3 and d4 is 25.0
```

Fig. 7.20 | Using variable-length argument lists. (Part 2 of 2.)

Wrapper Classes

- Java provides eight type-wrapper classes that enable primitive-type values to be converted to objects and vice versa.
  - Boolean
  - Character
  - Short
  - Byte
  - Integer
  - Long
  - Float
  - Double
- The wrapper classes do not have no-arg constructors.
- The instances of all wrapper classes are immutable, i.e., their internal values cannot be changed once the objects are created.

Numeric Wrapper Class Constructors

- You can construct a wrapper object either from a primitive data type value or from a string representing the numeric value.
- The constructors for Integer and Double are:
  - ✓ public Integer(int value)
  - ✓ public Integer(String s)
  - ✓ public Double(double value)
  - ✓ public Double(String s)

Conversion Methods

- Each numeric wrapper class implements the abstract methods which are defined in the Number class:

  - doubleValue,
  - floatValue,
  - intValue,
  - longValue, and
  - shortValue,

- These methods convert objects into primitive type values.

The Static valueOf Methods

- The numeric wrapper classes have a useful class method, valueOf(String s).
- This method creates a new object initialized to the value represented by the specified string.

For example:

```
Double doubleObject = Double.valueOf("12.4");

Integer integerObject = Integer.valueOf("12");
```

Parsing Strings into Numbers

- You have used the **parseInt** method in the Integer class to parse a numeric string into an int value and the **parseDouble** method in the Double class to parse a numeric string into a double value.
- Each numeric wrapper class has two overloaded parsing methods to parse a numeric string into an appropriate numeric value based on 10 (decimal) or any specified radix (e.g., 2 for binary, 8 for octal, and 16 for hexadecimal).

Example

- These two methods are in the Integer class
  - public static int parseInt(String s)
  - public static int parseInt(String s, int radix)
- Integer.parseInt("11", 2) returns 3;
- Integer.parseInt("12", 8) returns 10;
- Integer.parseInt("13", 10) returns 13;
- Integer.parseInt("1A", 16) returns 26;
- Integer.parseInt("12", 2) would raise a runtime exception because 12 is not a binary number.

Autoboxing and Unboxing

- JDK 1.5 allows primitive type and wrapper classes to be converted automatically. For example, the following statement in (a) can be simplified as in (b):

(a)

```
Integer[ ] intArray = {new Integer(2), new Integer(4)};
```

Equivalent

(b)

```
Integer[ ] intArray = {2, 4};
```

boxing

```
int [] intArray = {new Integer(1), new Integer(2)};
System.out.println(intArray[0] + intArray[1] );
```

Unboxing

The `String` Class

- Class String(package **java.lang**) is used to represent strings in Java.
- A string is an object of class String.
- String literals(stored in memory as String objects) are written as a sequence of characters in double quotation marks.
- Common string manipulation:
  - ✓ String Concatenation (concat)
  - ✓ Substrings (substring(index), substring(start, end))
  - ✓ Comparisons (equals, compareTo)
  - ✓ String Conversions
  - ✓ Obtaining String length and Retrieving Individual Characters in a string
  - ✓ Finding a Character or a Substring in a String
  - ✓ Conversions between Strings and Arrays
  - ✓ Converting Characters and Numeric Values to Strings

String Methods

- Some common string methods:

- **charAt(int index):** Returns the char value at the specified index.
- **compareTo(String anotherString):** Compares two strings lexicographically.
- **compareToIgnoreCase(String str):** Compares two strings lexicographically, ignoring case differences.
- **concat(String str):** Concatenates the specified

String Methods…

- **endsWith(String suffix):** Tests if this string ends with the specified suffix.
- **startsWith(String prefix):** Tests if this string starts with the specified prefix.
- **length():** Returns the length of this string.
- **isEmpty():** Returns true if, and only if, length() is 0.
- **matches(String regex):** Tells whether or not this string matches the given regular expression.
- **toString():** This object (which is already a string!) is itself returned.
- **toLowerCase():** Converts all of the characters in this String to lower case using the rules of the default locale.

Constructing Strings

- You can create a string object from a string literal or from an array of characters.
- To create a string from a string literal, use the syntax:

```
String newString = new String(stringLiteral);
```

Example:

```
String message = new String("Welcome to Java");
String s = new String();
```

- Since strings are used frequently, Java provides a shorthand initializer for creating a string:

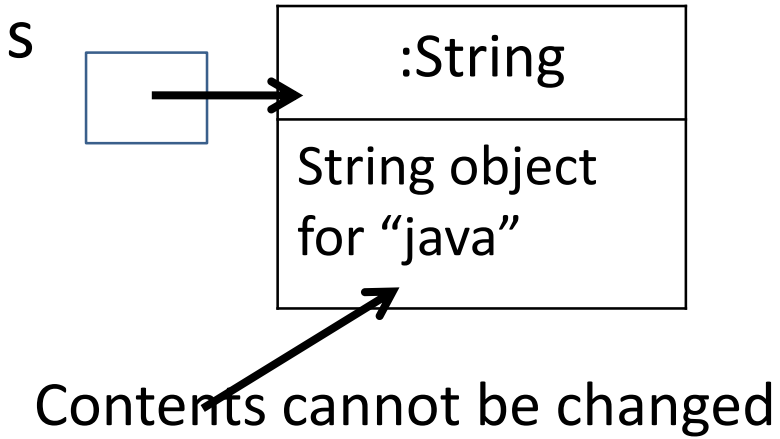
```
String message = "Welcome to Java";
```

Strings Are Immutable

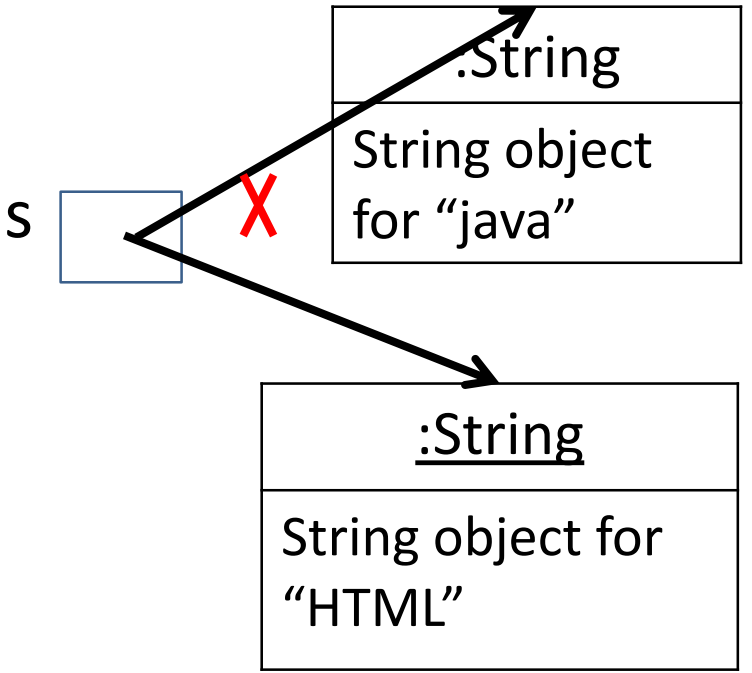
- A String object is immutable; its contents cannot be changed.
- Does the following code change the contents of the string?

```
String s = "Java";
s = "HTML";
```

After executing String s = “Java”;



After executing s = “HTML”;

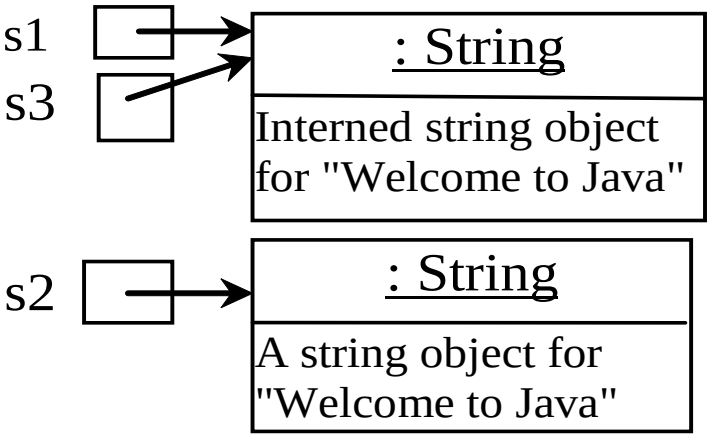


Interned Strings

- String interning is a method of storing only one copy of each distinct string value, which must be immutable.
- If you use the string initializer, no new object is created if the interned object is already created.
- A new object is created if you use the new operator.

Examples

```
String s1 = "Welcome to Java";
String s2 = new String("Welcome to Java");
String s3 = "Welcome to Java";
System.out.println("s1 == s2 is " + (s1 == s2));
System.out.println("s1 == s3 is " + (s1 == s3));
```



display

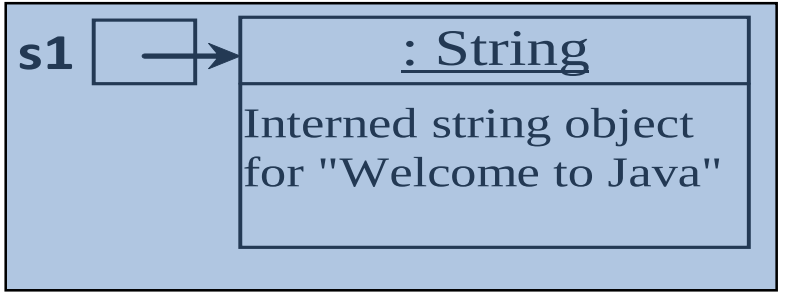
s1 == s2 is false

s1 == s3 is true

String s1 = "Welcome to Java";

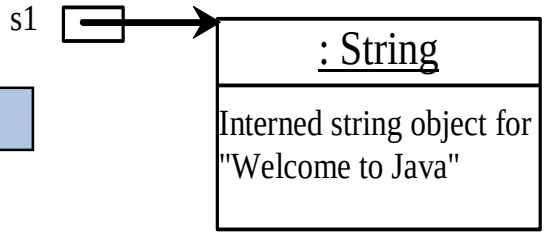
String s2 = new String("Welcome to Java");

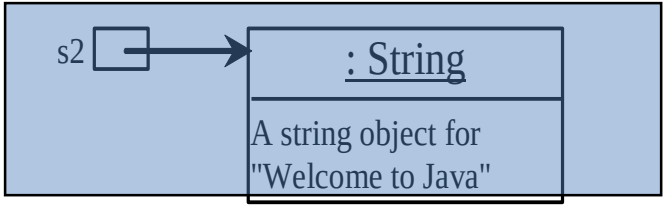
String s3 = "Welcome to Java";



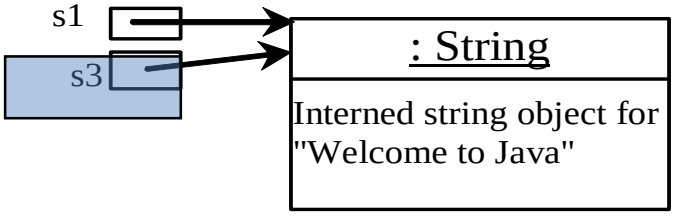
Cont'd…

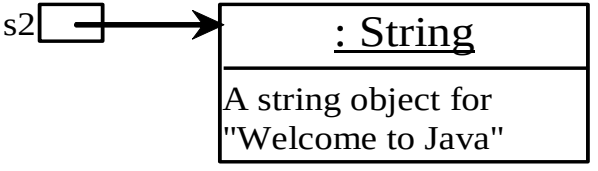
```
String s1 = "Welcome to Java";
String s2 = new String("Welcome to Java");
String s3 = "Welcome to Java";
```






```
String s1 = "Welcome to Java";
String s2 = new String("Welcome to Java");
String s3 = "Welcome to Java";
```





Convert Characters and Numbers to Strings

- The String class provides several static **valueOf** methods for converting a character, an array of characters, and numeric values to strings.
- These methods have the same name valueOf with different argument types char, char[], double, long, int, and float.

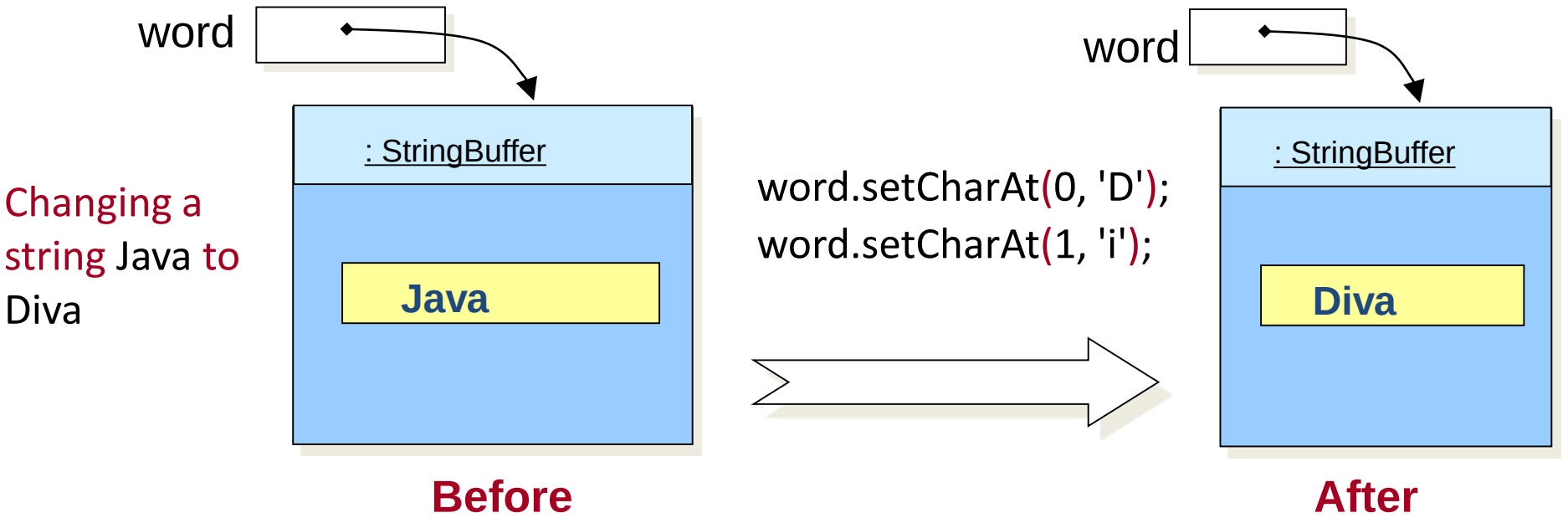
For example, to convert a double value to a string, use String.valueOf(5.44). The return value is string consists of characters ‘5’, ‘.’, ‘4’, and ‘4’.

StringBuilder and StringBuffer

- A **StringBuffer** is like a String, but can be modified.
- The length and content of **StringBuffer** can be changed through certain method calls.
- **StringBuilder** class provides an API compatible with StringBuffer, but with no guarantee of synchronization.
- Instances of StringBuilder are not safe for use by multiple threads. If such synchronization is required then it is recommended that StringBuffer be used.
- The principal operations on a StringBuilder and StringBuffer are the **append** and **insert** methods, which are overloaded so as to accept data of any type.
- **setCharAt** method can be used to set / change the character at the

StringBuffer Example

```
StringBuffer word = new StringBuffer("Java");
word.setCharAt(0, 'D');
word.setCharAt(1, 'i');
```



Summary

- In general, the StringBuilder and StringBuffer classes can be used wherever a string is used.
- StringBuilder and StringBuffer are more flexible than String.
- You can add, insert, or append new contents into StringBuilder and StringBuffer objects, whereas the value of a String object is fixed once the string is created.
- The StringBuilder class is similar to StringBuffer except that the methods for modifying the buffer in StringBuffer are synchronized, which means that only one task is allowed to execute the methods.
- Use StringBuffer if the class might be accessed by multiple tasks concurrently, because synchronization is needed in this case to prevent corruptions to StringBuffer.
- Using StringBuilder is more efficient if it is accessed by just a single task, because no synchronization is needed in this case.

Summary...

- The constructors and methods in StringBuffer and StringBuilder are almost the same.
- You can replace StringBuilder in all occurrences in this section by StringBuffer.
- The program can compile and run without any other changes.
- The StringBuilder class has three constructors and more than 30 methods for managing the builder and modifying strings in the builder

Regular Expressions

- You can match, replace, or split a string by specifying a pattern.
- A regular expression(abbreviated regex) is a string that describes a pattern for matching a set of strings.
- Regular expression is a powerful tool for string manipulations.
- Rules:
  - The brackets [ ] represent choices
  - The asterisk symbol * means zero or more occurrences.
  - The plus symbol + means one or more occurrences.
  - The hat symbol ^ means negation.
  - The hyphen – means ranges.
  - The parentheses ( ) and the vertical bar | mean a range of choices for multiple characters.

Regular Expression Examples

Expression	Description
[013]	A single digit 0, 1, or 3.
[0-9][0-9]	Any two-digit number from 00 to 99.
[0-9&&[^4567]]	A single digit that is 0, 1, 2, 3, 8, or 9.
[a-z0-9]	A single character that is either a lowercase letter or a digit.
[a-zA-z][a-zA-Z0-9_$]*	A valid Java identifier consisting of alphanumeric characters, underscores, and dollar signs, with the first character being an alphabet.
[wb](ad|eed)	Matches wad, weed, bad, and beed.
(AZ|CA|CO)[0-9][0-9]	Matches AZxx,CAxx, and COxx, where x is a single digit.

Matching, Replacing and Splitting Strings

- The matches, replaceAll, replaceFirst, and split methods can be used with a regular expression.

Examples:

"Java".matches("Java");

"Java is powerful".matches("Java.*")

"Welcome".replace('e', 'A') returns a new string, WAlcomA.
"Welcome".replaceFirst("e", "AB") returns a new string, WABlcome.
"Welcome".replace("e", "AB") returns a new string, WABlcomAB.
"Welcome".replace("el", "AB") returns a new string, WABcome.

String s = "Java Java Java".replaceAll("v\\w", "wi") ;
String s = "Java Java Java".replaceFirst("v\\w", "wi") ;

Splitting a String

- The split method can be used to extract tokens from a string with the specified delimiters.

For example, the following code:

```
String[] tokens = "Java#HTML#Perl".split("#");
for (int i = 0; i < tokens.length; i++)
  System.out.print(tokens[i] + " ");
```

Displays

Java HTML Perl